



Automatización de procesos con n8n

1º Administración de Sistemas Informáticos en Red

Víctor Palanca Pesudo

IES Juan de Garay

2026

Índice

1. Introducción
 - 1.1 Fundamentación teórica
 - 1.2 Contexto, motivación y objetivos generales
2. Metodología y herramientas empleadas
3. Talleres
 - 3.1 Taller 1: Pokemon Scraper
 - 3.1.1 Objetivo
 - 3.1.2 Requisitos previos
 - 3.1.3 Implementación
 - 3.1.4 Resultados
 - 3.1.5 Retos encontrados
 - 3.2 Taller 2: Wikipedia Agent
 - 3.2.1 Objetivo
 - 3.2.2 Requisitos previos
 - 3.2.3 Implementación
 - 3.2.4 Resultados
 - 3.2.5 Retos encontrados
 - 3.3 Taller 3: Sistema RAG Estándar
 - 3.3.1 Objetivo
 - 3.3.2 Requisitos previos
 - 3.3.3 Implementación
 - 3.3.4 Resultados
 - 3.3.5 Retos encontrados
 - 3.4 Taller 4: Telegram BOT
 - 3.4.1 Objetivo
 - 3.4.2 Requisitos previos
 - 3.4.3 Implementación
 - 3.4.4 Resultados
 - 3.4.5 Retos encontrados
4. Integración de los talleres
5. Conclusión
6. Bibliografía

1. Introducción

1.1 Fundamentación teórica

En nuestra sociedad, las personas realizamos constantemente tareas y procesos repetitivos. Concretamente, adentrándonos en el terreno empresarial, la eficiencia y la productividad muchas veces disminuyen debido a esto, ya que la repetición supone una pérdida enorme de tiempo, dinero y esfuerzo. Un ejemplo, son las pequeñas y medianas empresas en España, que pasan la mayor parte del tiempo buscando información, realizando informes, enviando correos electrónicos, copiando información a una hoja de cálculo y enviando avisos. Y ya no solo estas, incluso las grandes empresas sufren de esto. Necesitan automatizar tareas logísticas, financieras, y en empresas tecnológicas, gestionar la gran cantidad de datos y sistemas que manejan, además de reducir errores. Es ahí, donde las herramientas informáticas de automatización de tareas pueden ser realmente útiles.

Existen diversas herramientas para automatizar procesos, entre las que están Zapier, Make, UiPath, Microsoft Power Automate y n8n. Sin embargo, es en esta última en la que nos vamos a centrar ya que tiene multitud de ventajas.

La herramienta de automatización n8n es un modelo No-code de código abierto con licencia propietaria que permite conectar más de 400 diferentes aplicaciones (*Sass*) a través de sus API's oficiales, bases de datos mediante un sistema visual de nodos. A diferencia de los competidores comentados anteriormente, n8n no es solo una plataforma en la nube, sino un software que se puede instalar en un servidor propio. Esto es una gran ventaja debido a que los datos tienen un extra de seguridad al no tener que pasar por servidores externos y es más económico porque solo se incurre en gastos relacionados con la infraestructura del servidor. Además, una de las grandes características es que tiene agentes de IA integrados y personalizados, lo que permite crear chatbots, generar resúmenes o extraer información automáticamente. Por otra parte, tiene alta escalabilidad, puesto que procesa más de 200 tareas por segundo en una sola instancia.

En cuanto a su aplicación, esta se puede emplear en multitud de escenarios, desde gestión de contactos, sincronización de bases de datos, seguimiento de tareas, automatización de scripts, monitorización de procesos, y más.

Sin embargo, también tiene algunas desventajas, y es su curva pronunciada de aprendizaje, su número inferior de integraciones pre-configuradas que sus competidores, el mantenimiento

regular al alojarla en un servidor propio, y su mayor consumo de hardware debido a su alta velocidad de ejecución.

En cuanto a su funcionamiento, n8n se basa en la idea de que cada proceso puede segmentarse en pasos lógicos (nodos), conectados entre sí formando un flujo o *workflow*. Este flujo empieza con un *trigger* o disparador, como puede ser el recibir un correo nuevo, y a partir de entonces se ejecutan tareas de forma encadenada, como el análisis del mensaje, el almacenamiento de los datos y el envío de una notificación. Entre los nodos existentes, están los *triggers*, que acabamos de comentar, los nodos de acción que ejecutan una acción concreta, los nodos lógicos que controlan el flujo de la ejecución con condiciones y bucles, los nodos de datos (que permiten filtrar o analizar los datos) y los nodos personalizados, entre otros más..

Por otra parte, n8n mueve la información en formato JSON, un lenguaje universal que emplea objetos clave/valor y que usan las aplicaciones para hablar entre sí. Esto es muy importante ya que su ligereza hace que ocupe poco espacio de memoria y por tanto las automatizaciones sean más rápidas.

1.2 Contexto, motivación y objetivos generales

Todas estas funcionalidades y nuevas herramientas que están surgiendo han dado pie a que me anime a participar en un proyecto muy interesante basado en diferentes talleres de los cuales voy a aprender cómo realizar multitud de automatización de tareas, a entender la lógica de los nodos, manejar estructuras JSON y conectar diferentes API, entre otros. Esto da pie a adquirir nuevo conocimiento, desarrollando mi curiosidad y mis aptitudes a la hora de trabajar.

2. Metodología y herramientas empleadas

Como acabo de comentar, a lo largo de 2 meses se irán desarrollando 4 talleres, con dificultad incrementada y cada uno con una temática diferente. Además, se desarrollará esta memoria, que servirá de guía para demostrar el conocimiento que he adquirido y cómo he ido desarrollando cada tarea.

Entre las herramientas empleadas están la herramienta de automatización de n8n versión 1.x self-hosted o cloud, que esta última es la que voy a utilizar y la plataforma de Github, que servirá para alojar mi repositorio e ir adjuntando mis prácticas e ir consultando la información

sobre estas que se me ha facilitado. Además, será necesario Ollama una plataforma de código abierto diseñada para descargar y ejecutar Grandes Modelos de Lenguaje (LLMs), con los modelos llama 3.2 y llama 3.1 para embeddings. Y finalmente, una cuenta de Google Cloud para tener acceso a las APIs y herramientas de Google.

3. Talleres

3.1 Taller 1 - Pokemon Scraper

3.1.1 Objetivo

El objetivo de esta práctica es automatizar la consulta y el almacenamiento de datos de Pokémons. Aprenderemos cómo el sistema, a través de sus nodos, es capaz de leer una tabla de google sheet con los ID de los pokémons y, a través del flujo, se filtran los que aún no tiene datos, se consulta una API, se extrae la información relevante, se vuelve a escribir en el documento y envía una notificación al correo electrónico.

3.1.2 Requisitos previos

Antes que nada, ha sido necesario crearme una cuenta de Google Cloud para acceder tanto a la API de Google Sheets como a la de Gmail. Una vez creada la cuenta, configuré mi cuenta para enlazar cualquier servicio, puse la app en público, creé un nuevo proyecto llamado n8n-sheets y n8n-mail y saqué el OAuth client ID. Esta, una credencial única asignada a cada aplicación por un proveedor de servicios, y que permite que aplicaciones externas como n8n se identifiquen y soliciten permisos para acceder a servicios de terceros en mi nombre. Posteriormente, configuré las APIs en n8n.

Por otra parte, necesitaremos acceso a la API de Pokemon (PokéAPI), la cuál no requiere de registro ni de key, ya que es pública y gratuita. Esta tiene como límite 100 peticiones por minuto, suficiente para workflows pequeños.

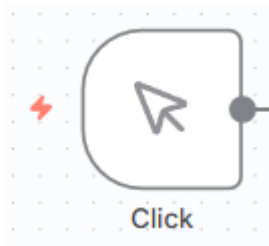
3.1.3 Implementación

En primer lugar, creé la hoja en blanco de n8n y luego el documento de google sheets con esta estructura, de manera que sólo estuviesen indicados los IDs de los Pokémon.

ID	Nombre	Tipo	Sprite Frontal	Sprite Posterior
1				
2				
4				
7				
15				
22				
32				
43				
53				
84				

- Nodo 1: Manual Trigger “Click”

En segundo lugar, configuré el primer nodo, el *trigger*, que en este caso va a ser un click manual. Este es perfecto para pruebas y se activa al pulsar *Execute workflow*.



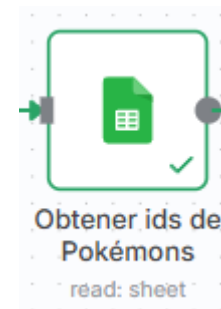
- Nodo 2: Google Sheets (Recuperar las filas del documento)

En segundo lugar, creé el nodo de Google Sheets con el objetivo de recuperar las filas del documento anteriormente creado. Para ello, vinculé mi documento y hoja al nodo y la configuré como “Get Rows” u obtener filas. En este caso no había ningún filtro adicional, por lo que no se tocó ninguna otra configuración. Como vemos, recupera el JSON con todas las filas y nombre de las columnas, y también podemos verlo en modo tabla.

Cabe destacar, que cada paso que voy haciendo en cada nodo voy haciéndole “pin” para guardar la información para el siguiente.

 The image shows the configuration interface for a Google Sheets node in a workflow editor. On the left, the 'Parameters' tab is active, showing settings for 'Credential to connect with' (Google Sheets account 6), 'Resource' (Sheet Within Document), 'Operation' (Get Row(s)), 'Document' (From list, taller1), 'Sheet' (From list, Hoja 1), 'Filters' (Currently no items exist), 'Combine Filters' (AND), and 'Options' (No properties). On the right, the 'OUTPUT' tab shows a JSON array of 10 items. The first three items are visible:


```
[
  {
    "row_number": 2,
    "ID": 1,
    "Nombre": "",
    "Tipo": "",
    "Sprite Frontal": "",
    "Sprite Posterior": ""
  },
  {
    "row_number": 3,
    "ID": 2,
    "Nombre": "",
    "Tipo": "",
    "Sprite Frontal": "",
    "Sprite Posterior": ""
  },
  {
    "row_number": 4,
    "ID": 4,
    "Nombre": "",
    "Tipo": "",
    "Sprite Frontal": "",
    "Sprite Posterior": ""
  },
  {
    "row_number": 5,
    "ID": 7,
    "Nombre": "",
    "Tipo": "",
    "Sprite Frontal": "",
    "Sprite Posterior": ""
  },
  ...
]
```



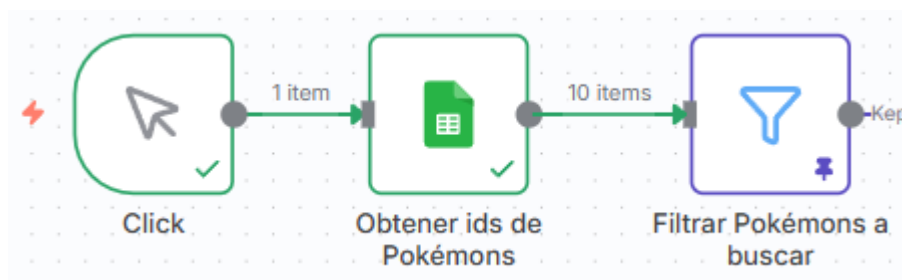
- **Nodo 3 - Filtrado de datos**

Este nodo, permite recuperar únicamente las filas en las que el Pokémon no tiene aún el nombre en el documento de Google Sheets, lo que indica que aún no ha sido procesado por la PokéAPI ni se ha recuperado su información. Esto ahorra espacio de memoria al no tener que volver a pedir todos los datos de nuevo.

En cuanto a su configuración, ponemos como condición que el nombre del Pokémon esté vacío (“is empty”). Basta con arrastrar el elemento para que se ajuste y como vemos, al ejecutar el paso, selecciona solo los que hemos indicado.

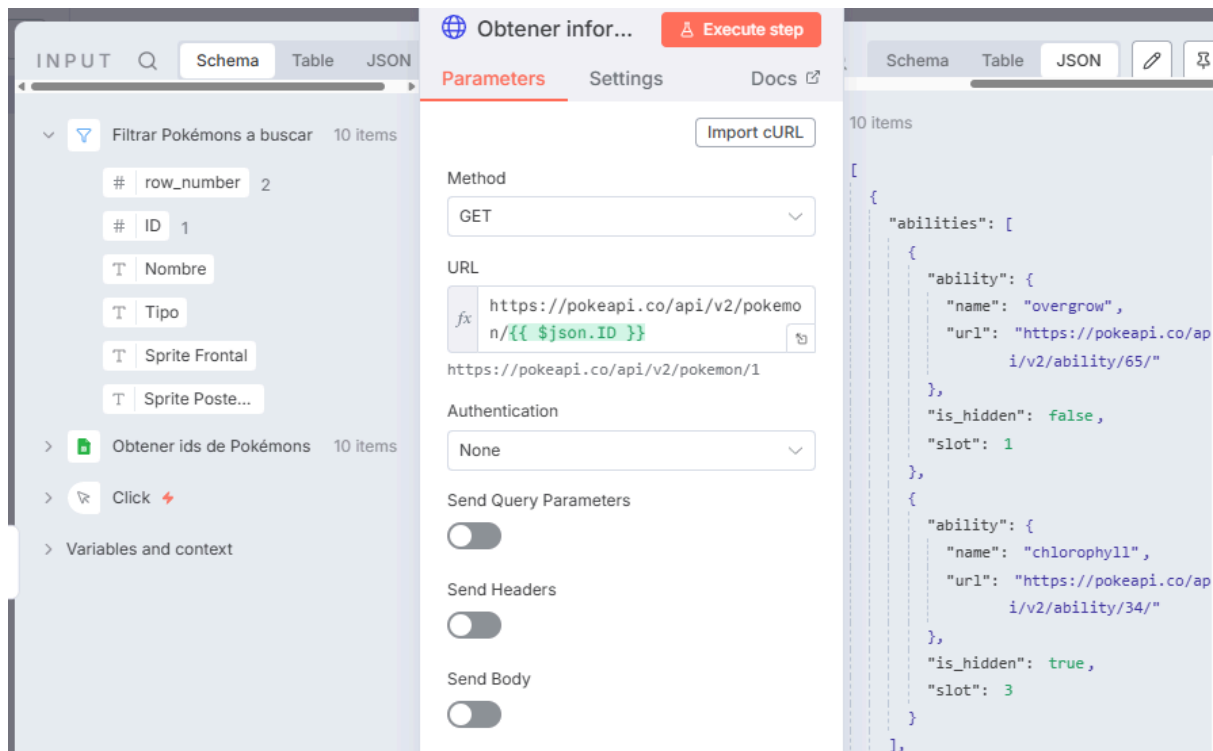
The screenshot shows the configuration of the 'Filtrar Pokémons a bu...' node. The 'Parameters' tab is selected, displaying a condition where the expression `fx {{ $json.Nombre }}` is set to 'is empty'. Below this, there are buttons for 'Add condition' and 'Add option'. The 'Options' section is currently empty. On the right, the 'OUTPUT' panel shows the results of the filter: 'Kept (10 items)' and 'Discarded'. The 'Kept' section displays a JSON array of 10 items, each with a 'row_number' and an 'ID', and empty strings for 'Nombre', 'Tipo', 'Sprite Frontal', and 'Sprite Posterior'.

Como podemos ver en el esquema, en cada línea entre nodo y nodo van saliendo los ítems que se van recuperando del nodo anterior.



- Nodo 4 - HTTP Request (Extraer Pokémons a buscar)

A continuación, con este nodo vamos a solicitar recuperar la información del servidor de la PokéAPI mediante el método GET de HTTPS. Para ello, conociendo la url de la API, la insertamos en el sitio indicado y arrastramos el elemento por el que busca la aplicación (puede ser por el nombre del Pokémon o por el ID pero nosotros vamos a optar por el segundo). Ponemos la autenticación en “none” ya que al ser una API pública no necesitamos ninguna “key”. No cambiamos ningún parámetro más.



Como vemos, el servidor nos ha devuelto todos los datos de los Pokémons que hemos solicitado, pero eso no es lo que queremos, ya que queremos unos datos en concreto, por lo que pasamos al siguiente nodo.

- Nodo 5 - Edit Fields (Extraer datos del Pokémon)

Este nodo sirve para filtrar de la información que hemos recuperado, los 5 campos que queremos para que estén listos para escribirse en el Google Sheets. Para ello, ponemos el modo en “Manual Mapping” para configurar los campos uno a uno usando la interfaz visual. Luego, ponemos un nombre identificativo al dato recibido, ponemos el tipo de dato y arrastramos del esquema el campo que queremos.

The screenshot displays the 'Extraer datos' (Extract Data) node in a workflow tool, configured in 'Manual Mapping' mode. The node is titled 'Extraer datos ...' and has an 'Execute step' button. The 'Parameters' tab is active, showing the 'Fields to Set' configuration.

Fields to Set Configuration:

- identificador:** Type: Number. Value: `{{ $json.id }}`. Example: 1.
- nombre:** Type: String. Value: `{{ $json.name }}`. Example: bulbasaur.
- tipo:** Type: String. Value: `{{ $json.types[0].type.name }}`. Example: grass.
- imagen frontal:** Type: String. Value: `{{ $json.sprites.front_default }}`. Example: <https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/back/1.png>.
- imagen trasera:** Type: String. Value: `{{ $json.sprites.back_default }}`.

INPUT Schema: The input schema shows a list of Pokémon abilities. The selected field is `abilities[0].ability.name`, which is mapped to the 'identificador' field.

OUTPUT: The output shows a JSON array of 10 items. The first item is:

```
{
  "identificador": 1,
  "nombre": "bulbasaur",
  "tipo": "grass",
  "imagen frontal": "https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/back/1.png",
  "imagen trasera": "https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/back/1.png"
},
```

- Nodo 6 - Google Sheets update (Actualizar información de los Pokémons)

Como en el segundo nodo, lo configuramos para que acceda al documento. Y en su configuración, ponemos que la operación sea actualizar las filas y “Match Each Column Manually” para decirle a la herramienta qué columna de la base de datos usar como identificador único para saber qué fila modificar. En este caso, los configuramos para que use el ID. Luego, ponemos los valores que queremos que actualice.

Actualizar inf... Execute step

Parameters Settings Docs

From list

Sheet: From list

Mapping Column Mode: Map Each Column Manually

Column to match on: ID

The column to use when matching rows in Google Sheets to the input items of this node. Usually an ID.

Values to Update

- ID (using to match): `{{ $json.identificador }}`
- Nombre: `{{ $json.nombre }}`
- Tipo: `{{ $json.tipo }}`
- Sprite Frontal: `{{ $json['imagen frontal'] }}`
- Sprite Posterior: `{{ $json['imagen trasera'] }}`

OUTPUT 10 items

```
[
  {
    "ID": 1,
    "Nombre": "bulbasaur",
    "Tipo": "grass",
    "Sprite Frontal": "https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/1.png",
    "Sprite Posterior": "https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/back/1.png"
  },
  {
    "ID": 2,
    "Nombre": "ivysaur",
    "Tipo": "grass",
    "Sprite Frontal": "https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/2.png",
    "Sprite Posterior": "https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/back/2.png"
  }
]
```

ID	Nombre	Tipo	Sprite Frontal	Sprite Posterior		
1	bulbasaur	grass	https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/1/1.png	https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/1/2.png		
2	ivysaur	grass	https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/2/1.png	https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/2/2.png		
4	charmander	fire	https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/4/1.png	https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/4/2.png		
7	squirtle	water	https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/7/1.png	https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/7/2.png		
15	beedrill	bug	https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/15/1.png	https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/15/2.png		
22	fearow	normal	https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/22/1.png	https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/22/2.png		
32	nidoran-m	poison	https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/32/1.png	https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/32/2.png		
43	oddish	grass	https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/43/1.png	https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/43/2.png		
53	persian	normal	https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/53/1.png	https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/53/2.png		
84	doduo	normal	https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/84/1.png	https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/84/2.png		

Diagrama de flujo de un pipeline de datos en Airflow:

- Click** (Trigger)
- Obtener ids de Pokémons** (Tarea: read: sheet)
- Filtrar Pokémons a buscar** (Tarea: Kept)
- Obtener información Pokémon** (Tarea: 1 item)
- Extraer datos del Pokémon** (Tarea: manual)
- Actualizar información de Pokémons** (Tarea: update: sheet)

- Como queremos que posteriormente nos envíe un email cuando la operación se haya finalizado con los elementos actualizados vamos a agregarlos en uno solo para que no se envíe un correo por cada ítem. Para ello configuramos la agrupación únicamente con los campos que queremos y como vemos quedan agrupados.

11

- Nodo 8 - Gmail (Enviar un correo de resultado)

Finalmente, el flujo envía un correo de resultado indicando que se ha completado la ejecución satisfactoriamente. Para ello, configuramos el elemento como un mensaje, con la operación enviar, la dirección de correo a la que enviar la información, el asunto, el tipo de mensaje y el mensaje (junto con las etiquetas JSON).

The screenshot shows the configuration for the 'Enviar correo ...' node. The left pane displays a table of 11 items, grouped by 'ID' and 'Nombre'. The right pane shows the configuration for sending an email via Gmail.

ID	Nombre
ID[0]	1
ID[1]	2
ID[2]	4
ID[3]	7
ID[4]	15
ID[5]	22
ID[6]	32
ID[7]	43
ID[8]	53
ID[9]	84
ID[10]	85

The right pane configuration includes:

- Credential to connect with:** Gmail account 7
- Resource:** Message
- Operation:** Send
- To:** vppinf2@gmail.com
- Subject:** Actualización pokemon
- Email Type:** HTML
- Message:**

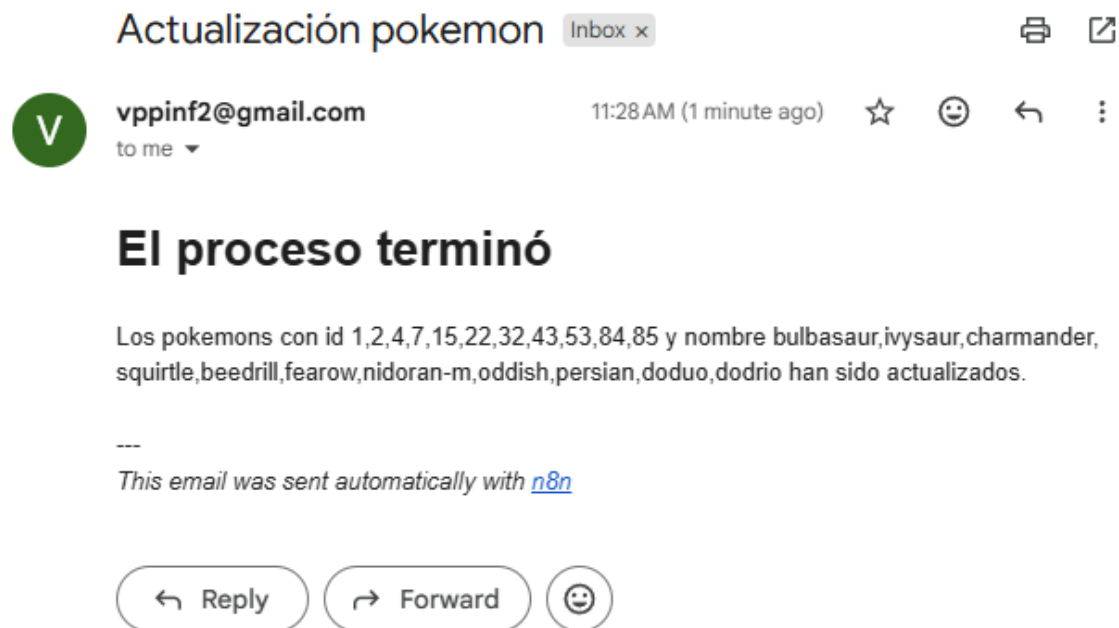
```
<h1> El proceso terminó </h1>
Los pokemons con id {{ $json.ID }} y nombre {{ $json.Nombre }} han sido actualizados.
```
- Options:** (empty)

3.1.4 Resultados finales

Finalmente, el flujo quedaría de la siguiente manera:



Y al ejecutarlo el correo se ha enviado con todos los datos correctamente:



3.1.5 Retos encontrados

Para comenzar, ha sido realmente fácil configurar este flujo. Sin embargo, lo más costoso ha sido conseguir las API keys de Google puesto que requerían suscripción y registro, y no funcionaba de forma correcta hasta que se iniciaba correctamente.

3.2 Taller 2: Wikipedia Agent

3.2.1 Objetivo

El objetivo de este taller es crear un flujo con 7 nodos (2 de ellos deshabilitados) a través del cuál un agente de IA llamado “Ana” es capaz de responder preguntas en tiempo real a través de la herramienta de Wikipedia. La memoria que tiene el agente es de 10 mensajes y podemos escoger entre tres modelos de lenguaje o LLM (Openrouter que es gratuito, OpenAI que es de pago, y Google Gemini que es gratuito con límites). Este chat estará disponible a través de un index.html personalizado.

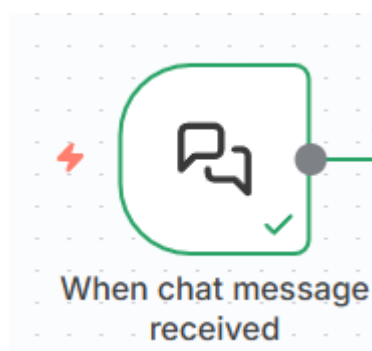
3.2.2 Requisitos previos

Son breves los requisitos previos en este caso. He conseguido las API keys de cada uno de los LLM que voy a emplear y los he insertado en la interfaz de n8n para poder tener acceso a la hora de hacer preguntas y que me puedan responder. Para ello, he tenido que acceder a cada una de las páginas de las IA’s y registrarme.

3.2.3 Implementación

- Nodo 1 - Chat Trigger “When chat message received” (Webhook público)

En primer lugar, el primer nodo es un webhook público, que es un concepto de conversación contrario al de las API. Una API, funciona solicitando cada cierto tiempo la información para comprobar o pedir nuevos datos, sin embargo, con los webhooks la información es recibida en el momento en el que hay una novedad, sin solicitarlo. En este caso, se utiliza un webhook público, que es un link que estará embebido en el index.html y dará acceso público al chat que tenemos internamente en el flujo de n8n (@n8nchat) para poder realizar preguntas. Cada mensaje será un input para el agente interno.



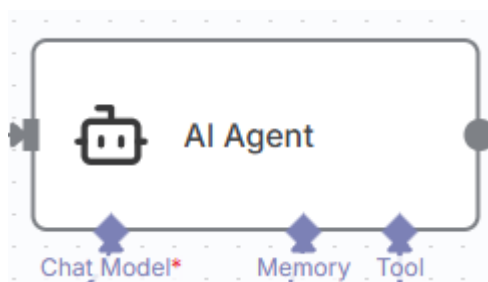
Como podemos observar en la imagen, ponemos el chat en embebido y sin autenticación ya que es público. Para que funcione realmente chat en el index.html hay que cambiar poner nuestro link en el apartado de webhookUrl y activar el flujo, que pone la URL en modo producción, de forma que esta dirección se queda escuchando constantemente.

The image shows the configuration for a chat workflow in n8n. The workflow is named "When chat message received". The "Parameters" tab is active, showing the "Chat URL" as `https://vps-86c1c2aa.vps.ovh.net/webhook/2a2a7435-a2c1-4718-8e81-6e563529e54b/chat`. The "Settings" tab shows "Make Chat Publicly Available" is turned on, "Mode" is set to "Embedded Chat", and "Authentication" is set to "None". A "Test chat" button is visible. To the right is a preview of the chat interface titled "WikiBot". It shows a welcome message: "Empieza una conversación. Estamos aquí para ayudarte 24/7." and a greeting: "Hola! 🤖". Below that is a message: "Me llamo Ana, soy un asistente de Wikipedia". At the bottom is a text input field with the placeholder "Escribe tu pregunta.." and a send button.

```
webhookUrl:
  'https://vps-86c1c2aa.vps.ovh.net/webhook/2a2a7435-a2c1-4718-8e81-6e563529e54b/chat',
```

- Nodo 2 - IA Agent (Agente principal)

Este nodo es el agente de IA encargado de procesar toda la información. Incorpora un prompt de sistema en el que se le da una identidad, un objetivo y unas herramientas para que sepa cómo funcionar.



System Prompt configurado (“Ana, asistente de Wikipedia”)

Objetivo

Eres un asistente amable, tu nombre es Ana, ayudas a buscar información en Wikipedia.

Reglas

- Devuelve siempre respuestas de dos párrafos o menos.
- Siempre pon las referencias de los artículos que consultas en Wikipedia.

Tools

- Tienes una "tool" para conectarte a Wikipedia y traer información, puedes usarla para confirmar la información

Luego, se le asigna un modelo, cada uno un nodo. En este caso he incorporado 3:

- OpenRouter (el que queda fijo) con el modelo concreto “nvidia/nemotron-nano-9b-v2:free”, ya que es gratuito.
- Gemini con el modelo “gemini-2.5-flash”, con 5 solicitudes por minuto, 250k tokens por minuto y 20 solicitudes máximo al día. Está deshabilitado en el flujo.

Modelo	Categoría	RPM ↓	TPM	RPD	G
Gemini 2.5 Flash	Modelos de texto de salida	0 / 5	0 / 250K	0 / 20	

- OpenAI con el modelo “gpt-4.1-mini”, de pago. Está deshabilitado en el flujo.

A continuación, le he asignado una ventana de memoria de 10 mensajes, para que pueda recordar lo comentado hasta 10 mensajes anteriores.

Simple Memory

Parameters

Settings

Docs

Session ID

Connected Chat Trigger Node

Session Key From Previous Node

fx

{{ \$json.sessionId }}

Context Window Length

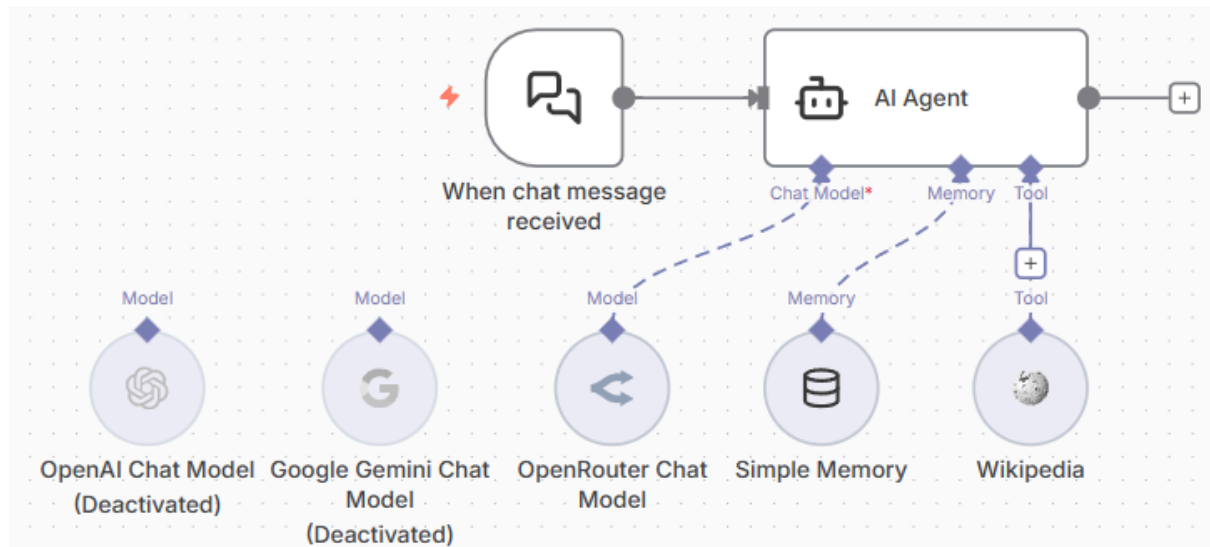
10

How many past interactions the model receives as context

Finalmente, le he asignado la herramienta de Wikipedia, que no requiere configuración ninguna.

3.2.4 Resultados

En la imagen siguiente podemos ver cómo queda el flujo de trabajo finalizado.

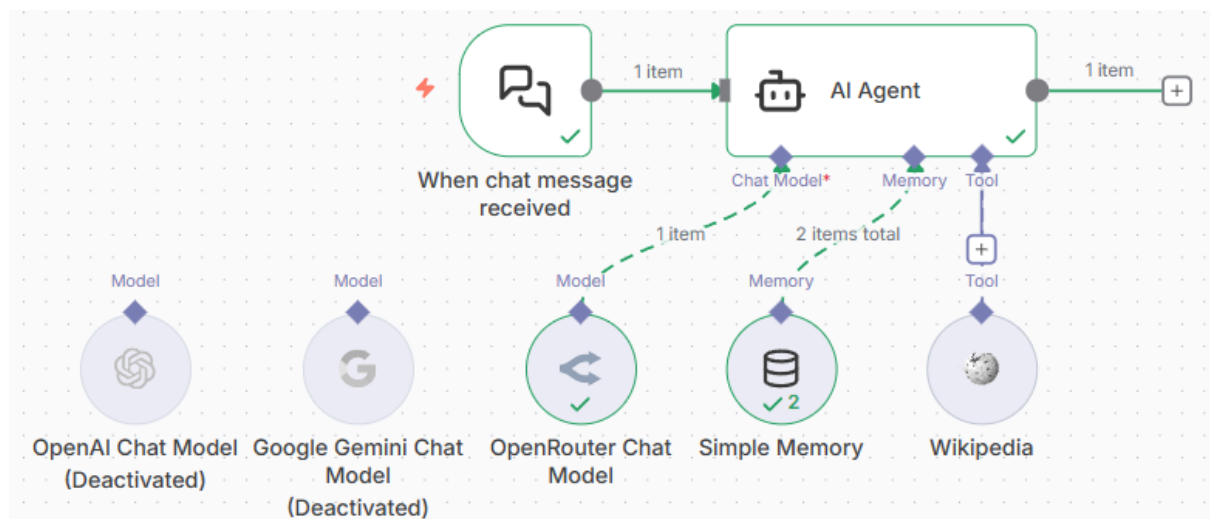


Ahora, podemos realizar preguntas al agente tanto desde el chat interno como desde el chat embebido en la página web (index.html).

1. Chat interno

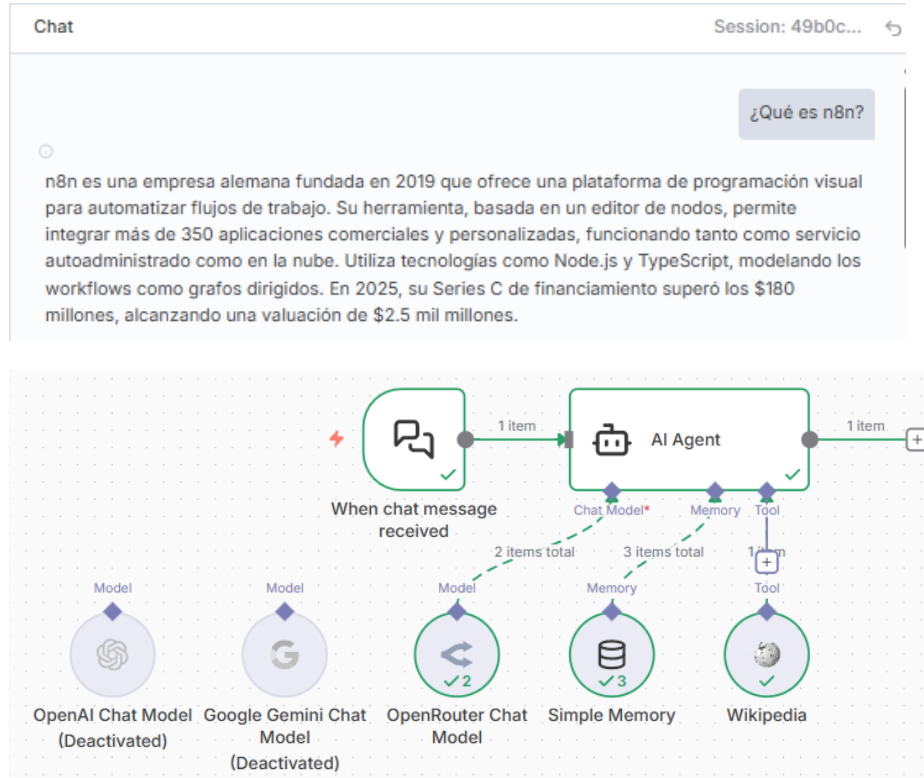
a) Opción 1: Vamos a realizarle una pregunta simple como cuál es su nombre (aunque también podríamos preguntarle algo de cultura general ya que la propia IA tiene guardada cierta información).

Como podemos ver no accede a la Wikipedia ya no tiene necesidad.



b) Opción 2: Vamos a realizarle una pregunta más compleja para que tenga que buscar información.

Podemos ver cómo en este caso, sí que accede a la wikipedia ya que no tiene los recursos en su sistema.



2. Chat externo

Podemos ver cómo funciona a la perfección y nos pone las referencias.

WikiBot
Empieza una conversación. Estamos aquí para ayudarte 24/7.

¿Qué es n8n?

n8nes una empresa alemana fundada en 2019 que desarrolla una plataforma de automatización de flujos de trabajo basada en una interfaz visual de nodos. Su herramienta permite integrarse con más de 350 aplicaciones, tanto comerciales como propias, y

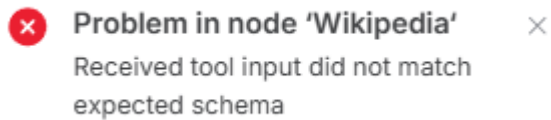
Escribe tu pregunta..

(Referencias: Wikipedia, TechCrunch)

3.2.5 Retos encontrados

En este caso, han sido varios los retos encontrados.

En primer lugar, tuve que cambiar Ollama por OpenRouter ya que este primero daba errores. La mayoría de sus versiones no era compatible con n8n y además, daba error a la hora de acceder a Wikipedia, ya que el esquema que generaba para acceder a la API de Wikipedia no era compatible y además funcionaba muy lento. Consideré que esta segunda opción era mejor.



En segundo lugar, tuve problemas a la hora de que apareciese la opción de escribir en el chat externo embebido en el html. Tuve que quitar y poner repetidamente el trigger para que me cambiara de URL para que me funcionara.

3.3 Sistema RAG Estándar

3.3.1 Objetivo

El objetivo de este taller es crear un asistente inteligente que consulta documentos PDF propios a través de un sistema RAG (Retrieval-Augmented-Generation), que consiste en dos fases diferenciadas.

- Fase 1 (Alimentación de la base de datos): Los PDF/documentos que adjuntamos se fragmentan en chunks, que son partes mínimas, y luego se vectorizan con embeddings y, finalmente, estos valores se guardan en el vector store.

Un embedding vectorial se puede definir como una representación numérica de datos (texto, imágenes, audio) en forma de vector de N dimensiones, donde elementos semánticamente similares ocupan posiciones cercanas en el espacio vectorial.

- Fase 2 (Consumo de base de datos): El individuo hace una pregunta por el chat, el agente vectoriza la pregunta busca chunks similares en el store anterior, los usa como contexto y genera una respuesta con referencia al documento.

3.3.2 Requisitos previos

En cuanto a los requisitos previos en este caso no hay ninguno. Sin embargo, es necesaria la lectura adecuada del manual y de la información para comprender los conceptos.

3.3.3 Implementación

FASE 1: INGESTIÓN

- Nodo 1 - On form submission (Form Trigger)

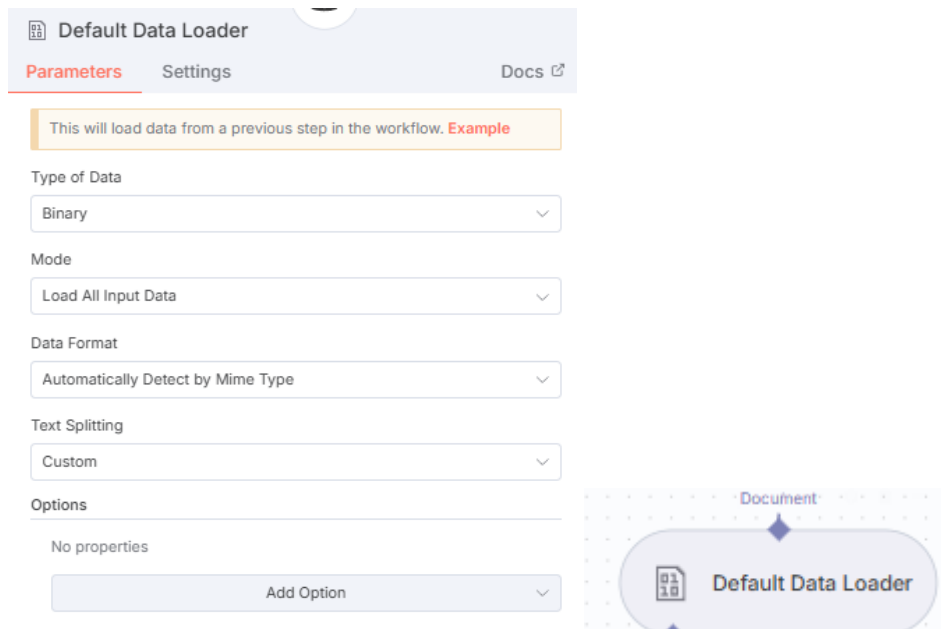
Creamos un formulario para poder adjuntar archivos o cargar documentos al vector store en este workflow.

The image shows the configuration for the 'On form submission' node in n8n. The configuration is divided into three tabs: 'Parameters', 'Settings', and 'Docs'. The 'Parameters' tab is active, showing the 'Form URLs' section with a 'Test URL' and a 'Production URL'. The 'Settings' tab shows the 'Form Elements' section, where the 'Field Name' is 'Documentos', the 'Element Type' is 'File', and the 'Multiple Files' option is enabled. The 'Accepted File Types' are set to '.pdf'. The 'Form Title' is 'Cargar documentos' and the 'Form Description' is 'e.g. We'll get back to you soon'. The 'Docs' tab is empty. To the right of the configuration is a preview of the generated form, which has a title 'Cargar documentos', a label 'Documentos', a file upload button 'Elegir archivos', and a 'Submit' button. The form is automated with n8n.

Hay que poner que únicamente acepte pdf's y, simplemente ponemos la descripción de la ventana emergente.

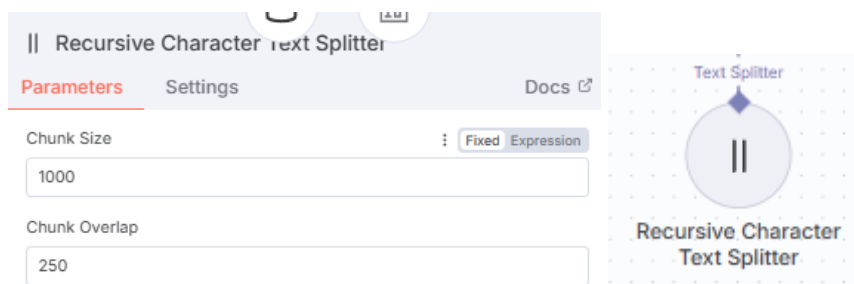
- **Nodo 2 - Default Data Loader (Cargador de documentos)**

Este nodo recibe los datos binarios del formulario (ya que se lo hemos especificado). Le ponemos en la configuración que detecte el archivo que le insertamos.



- **Nodo 3 - Recursive Character Text Splitter**

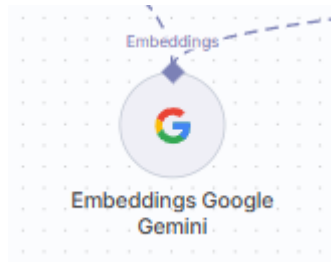
Este nodo, conectado al anterior, divide los documentos en fragmentos (chunks) de un tamaño manejable para poder generar los embeddings. Especificamos el tamaño del chunk y el overlap (lo ponemos a 250 para que el contexto no se corte bruscamente en el límite de dos chunks)



- **Nodo 4 - Nodo embedding (Google Gemini) - Compartido por las dos fases**

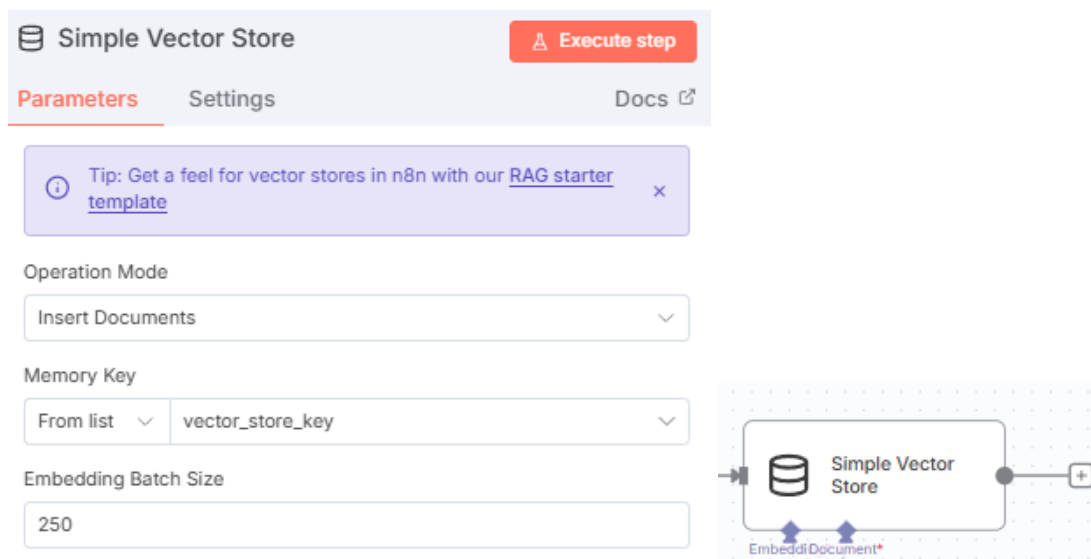
Este nodo convierte los chunks de texto en vectores numéricos (embeddings). El nodo es compartido por las dos fases, ya que en la primera vectoriza los chunks antes de guardarlos y en la segunda vectoriza la pregunta para buscar chunks similares por proximidad geométrica. En este caso, se pedía utilizar Ollama pero tras constantes errores he optado por Gemini, ya que es ciertamente gratuita y funciona perfectamente.

He utilizado el modelo “gemini-embedding-001” ya que solo trabajamos con texto en este caso y vamos a realizar búsquedas semánticas. Los otros modelos servirían si trabajáramos con imágenes, audio o video.



- Nodo 5 - Simple Vector Store (Memoria de embeddings)

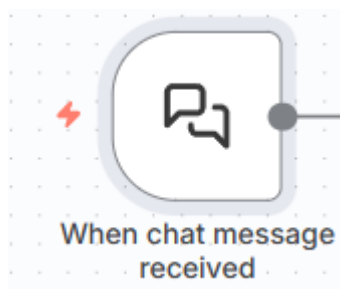
Este nodo almacena en memoria los embeddings generados. Esta memoria es volátil, por lo que si reiniciamos el flujo habría que volver a ejecutar la primera fase.



FASE 2: CONSUMO

- Nodo 6 - Trigger (When chat message received)

Es un webhook de chat que recibe las preguntas que le hacemos de forma interna. No configuramos nada más.



- Nodo 7 - AI Agent

Este nodo va a ser el servicio inteligente que gestione la entrada y salida de respuestas en base a la similitud semántica con el documento adjuntado.

Para que funcione como queremos, le ponemos el siguiente prompt:

Rol: Asistente Legal
Descripcion Eres un asistente legal para ayudar a un abogado a consultar documentos legales internos. Respuestas precisas, claras y fundamentadas. Tono profesional y formal. Respuestas cortas y puntuales. Si no conoces la respuesta, indica que no dispones de la informacion.
Capacidades - Búsqueda y recuperacion de informacion legal relevante. - Respuestas concisas con fundamento en los documentos. - Inclusion de referencia del documento consultado. - Identificacion de ausencia de informacion. - Confidencialidad y precision en todas las respuestas
Comportamiento 1. Recepcion de la consulta del abogado. 2. Búsqueda en la base de conocimiento. 3. Si hay informacion: resumen conciso + referencia (nombre de archivo, cláusula). Si no hay informacion: 'No puedo encontrar la respuesta en los recursos disponibles.' 4. Confidencialidad total. 5. Tono profesional, formal, claro y directo

Este nodo está conectado a los nodos 8, 9 y 10 que vemos a continuación:

- Nodo 8 (LLM) - Groq

Es el LLM conectado que va a elaborar la respuesta.

- Nodo 9 - Memoria conectada

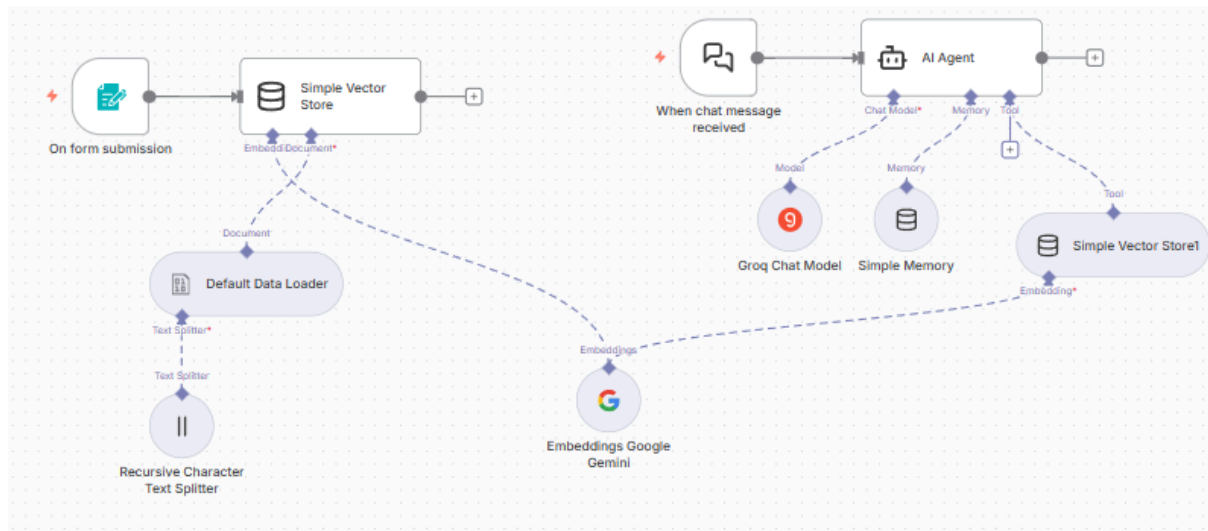
Ponemos 10 mensajes para que el LLM recuerde los últimos mensajes y que tenga suficiente contexto.

- Nodo 10 - Simple Vector Store1

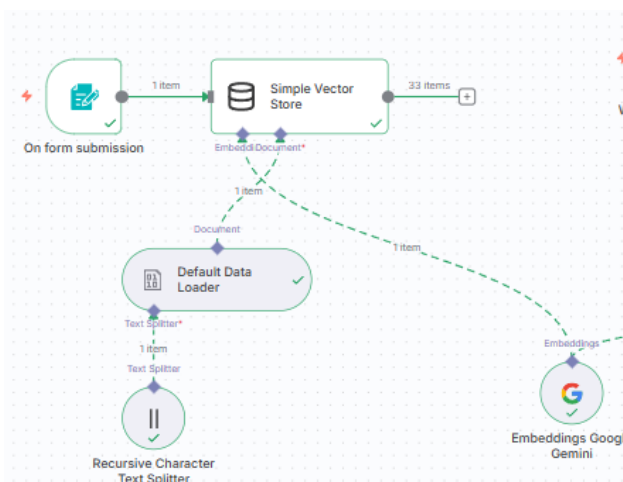
Es la base de datos en la que se guardan los vectores generados en la primera fase y está conectada a Gemini Embedding para que se traduzca la pregunta del usuario en vectores y posteriormente se puedan comparar los valores del documento con la pregunta.

3.3.4 Resultados

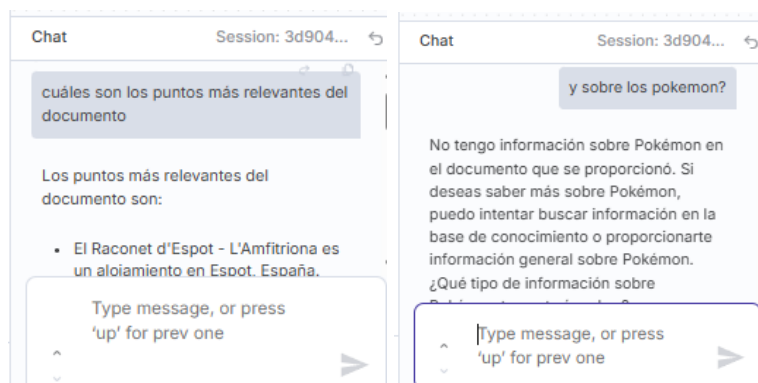
A continuación podemos ver el flujo de trabajo:



Cuando adjuntamos un pdf sobre una sentencia, por ejemplo, se cargan los datos binarios de este y se dividen en chunks. Vemos como en el proceso se ha activado también el embedding de Gemini y se ha almacenado en la base de datos de vectores.



Ahora le podemos hacer una pregunta sobre el documento y nos contesta. Si no lo sabe o no entiende la pregunta actúa en base al prompt que le hemos configurado.



3.3.5 Retos encontrados

En cuanto a los retos encontrados en este caso ha sido que Ollama daba constantes errores a la hora de realizarle preguntas y de hacer los vectores embedding, es por eso por lo que he cambiado para hacer los embeddings a Gemini. Y para el LLM he puesto a Groq ya que tenía más tokens y Gemini me daba error al sobrepasarlos.

3.4

6. Bibliografía

<https://www.nocodehackers.es/herramientas-no-code/n8n>

<https://www.iebschool.com/hub/que-es-n8n-y-como-automatizar-tareas/>

<https://ikaue.com/blog-data/que-es-n8n-y-que-ventajas-tiene-sobre-otras-herramientas-de-automatizacion>

<https://codigoelectronica.com/blog/tipos-de-nodos-en-n8n>

<https://www.redhat.com/es/topics/automation/what-is-a-webhook>

<https://www.javadex.es/blog/que-es-embedding-vectorial-como-funciona-guia-completa-202>

6